

Chapter 2 – Application Layer (Notes)

1. Introduction to the Application Layer

- **Role of the Application Layer**
 - This is the **topmost layer** of the Internet protocol stack.
 - It provides **services directly to the user's applications** such as web browsers, email clients, messaging apps, streaming platforms.
 - Without this layer, the Internet would just be raw data transfer with no usable services.
 - **Examples of Application-Layer Services:**
 - Social Networking (Facebook, Twitter, Instagram).
 - Web Browsing (Chrome, Firefox using HTTP/HTTPS).
 - E-Mail (Gmail, Outlook using SMTP, IMAP, POP3).
 - Messaging (WhatsApp, Messenger).
 - Streaming (YouTube, Netflix, Spotify).
 - Online Gaming (Fortnite, PUBG).
 - Remote Login (SSH).
 - Search Engines (Google).
 - **Key Focus Areas in Application Layer Study:**
 1. **Principles of network applications** (design models like client-server and P2P).
 2. **Protocols** such as HTTP, SMTP, DNS.
 3. **Programming tools** (sockets for TCP/UDP).
-

2. Principles of Network Applications

Creating a Network Application

- To build a networked app (e.g., web browser + server):
 - You only need to write code for **end systems** (client and server programs).
 - No need to touch routers or network core devices (they just forward packets).
 - This makes app development faster and widely deployable.
-

Client-Server Paradigm

- **Server:**
 - Always-on host with a fixed (often static) IP.
 - Usually hosted in data centers for reliability and scalability.
 - Examples: Web servers, email servers.
- **Clients:**
 - Devices like laptops or smartphones.
 - May have dynamic IP addresses, may disconnect and reconnect.

- Initiate communication but usually don't talk directly to each other.
 - **Examples:** HTTP (web), IMAP (mail), FTP (file transfer).
-

Peer-to-Peer (P2P) Paradigm

- **Definition:** No always-on server; peers communicate directly.
 - **How it works:**
 - Peers act both as clients and servers.
 - Example: BitTorrent (file sharing).
 - **Advantages:**
 - **Scalability:** New peers bring new resources as well as demands.
 - **Challenges:**
 - Peers come and go (churn), IP addresses change, management is complex.
 - **Examples:** BitTorrent, early Skype, blockchain networks.
-

Processes and Communication

- **Process:** A running program on a host.
 - **Communication:**
 - On the same host → use OS inter-process communication.
 - On different hosts → exchange **messages** across the network.
 - **Roles:**
 - **Client Process:** Initiates communication.
 - **Server Process:** Waits to be contacted.
-

Sockets

- **Definition:** A socket is the interface between the application process and the network.
 - **Analogy:** Like a door — the app pushes data into the socket, and the transport layer delivers it.
 - **Two sockets** are involved: one at sender, one at receiver.
-

Addressing Processes

- **Requirement:** To communicate, a process needs an identifier.
- **Identifiers include:**
 - **IP Address** (of host).
 - **Port Number** (of process on that host).
- **Examples of port numbers:**
 - HTTP → 80.
 - HTTPS → 443.
 - SMTP (mail) → 25.

Application-Layer Protocols

- Define:
 1. Types of messages (request/response).
 2. Message syntax (fields, order).
 3. Semantics (meaning of each field).
 4. Rules (when/how messages are sent).
 - **Open Protocols:** Published in RFCs (e.g., HTTP, SMTP). Anyone can use them.
 - **Proprietary Protocols:** Owned by companies (e.g., Skype, Zoom).
-

Transport Service Needs of Apps

Applications differ in requirements:

1. **Data Integrity:** File transfer needs 100% reliability; audio may tolerate some loss.
 2. **Timing:** Real-time apps (VoIP, gaming) need low latency.
 3. **Throughput:** Streaming video needs guaranteed minimum throughput; file transfers can adapt.
 4. **Security:** Encryption, authentication.
-

Internet Transport Services

- **TCP:** Reliable, connection-oriented, provides flow/congestion control. Used for web, email, file transfer.
 - **UDP:** Unreliable, connectionless, lightweight. Used for streaming, gaming where speed > reliability.
-

3. Web and HTTP

- **What is HTTP?**
 - HyperText Transfer Protocol.
 - Application-layer protocol of the web.
 - Uses client-server model: browser = client, Apache/Nginx = server.
 - Runs over TCP (port 80 or 443 with TLS).
- **HTTP Characteristics:**
 - Stateless: server doesn't remember previous requests.
 - Two connection types:
 1. **Non-Persistent:** New TCP connection for each object (inefficient).
 2. **Persistent:** Single TCP connection for multiple objects (HTTP/1.1).
- **Request Methods:**
 - **GET:** Retrieve object.
 - **POST:** Send form data in request body.

- **HEAD:** Retrieve headers only.
 - **PUT:** Upload/replace object.
 - **Response Codes:**
 - 200 OK → success.
 - 301 Moved Permanently → new location.
 - 400 Bad Request → syntax error.
 - 404 Not Found → page doesn't exist.
 - 505 → HTTP version unsupported.
 - **Cookies:**
 - Mechanism to maintain state in stateless HTTP.
 - Stored on client, sent back with each request.
 - Uses: login sessions, shopping carts, tracking users.
 - **Caching:**
 - Proxy caches reduce delay and bandwidth usage.
 - Browser uses **conditional GET** to avoid downloading unchanged objects.
 - **HTTP/2 & HTTP/3:**
 - HTTP/2 (2015): Multiplexing, server push, prioritization → reduced delay.
 - HTTP/3: Runs over QUIC (UDP) with encryption, removes TCP head-of-line blocking.
 - **QUIC:** A UDP-based transport protocol with built-in TLS encryption, faster handshakes, multiplexed streams, and support for connection migration.
-

4. Email and Protocols (SMTP, IMAP, POP3)

- **Components:**
 1. **User Agents (UA):** Mail clients (Outlook, Gmail app).
 2. **Mail Servers:** Store mailboxes and outgoing queues.
 3. **SMTP:** Transfers mail between servers.
 - **SMTP (Simple Mail Transfer Protocol):**
 - Uses TCP (port 25).
 - Push protocol (sender pushes mail to receiver's server).
 - Persistent connections.
 - Messages must be ASCII text.
 - **Mail Access Protocols:**
 - **IMAP:** Keeps mail on server; supports folders, multiple devices.
 - **POP3:** Downloads and deletes mail (simpler).
 - **HTTP-based:** Gmail, Yahoo Mail (web interface).
 - **Message Format:**
 - Defined by RFC 2822.
 - **Header:** To, From, Subject.
 - **Body:** Message text.
-

5. DNS (Domain Name System)

- **Purpose:** Maps human-readable names (e.g., google.com) to IP addresses.
 - **Why not centralize?** Single point of failure, too much traffic, poor scalability.
 - **Services Provided:**
 - Hostname → IP address translation.
 - Host aliasing (canonical names vs. easy aliases).
 - Mail server aliasing.
 - Load distribution (replicated servers).
 - **Hierarchy:**
 - **Root servers** → **TLD servers** (.com, .edu, .org, country domains) → **Authoritative servers** (organization's servers).
 - **Local DNS servers:** Provided by ISP, act as first point of contact.
 - **Queries:**
 - **Iterative:** Each server refers client to another server.
 - **Recursive:** Server does all work and returns final answer.
 - **DNS Records:**
 - **A:** Name → IP.
 - **NS:** Domain → authoritative server.
 - **CNAME:** Alias → canonical name.
 - **MX:** Mail server mapping.
 - **Security Issues:**
 - DNS cache poisoning (false IPs injected).
 - DDoS on root/TLD servers.
 - Solution: DNSSEC (authentication, integrity).
-

6. Peer-to-Peer Applications

- **BitTorrent Example:**
 - File divided into chunks.
 - Tracker helps peers find each other.
 - Peers exchange chunks (tit-for-tat strategy).
 - "Rarest first" download strategy.
 - Peers may leave (selfish) or stay to seed (altruistic).
 - **Comparison to Client-Server:**
 - Client-Server → server bottleneck, time grows with number of clients.
 - P2P → scalable, time decreases as more peers join (since they contribute upload capacity).
-

7. Video Streaming & Content Distribution Networks (CDNs)

- **Challenges:**
 - High demand (YouTube/Netflix consume 80% of ISP bandwidth).
 - Heterogeneity (different devices, speeds, networks).

- Continuous playback constraint → buffering needed.
 - **DASH (Dynamic Adaptive Streaming over HTTP):**
 - Video split into chunks encoded at multiple bitrates.
 - Client requests appropriate chunk based on bandwidth.
 - Adaptive → prevents buffering during congestion.
 - **Content Distribution Networks (CDNs):**
 - Store copies of videos across multiple locations.
 - Reduce delay by serving users from nearby servers.
 - **Akamai:** 240,000 servers worldwide.
 - **Netflix OpenConnect:** Netflix places its own CDN servers at ISPs.
-

8. Socket Programming

- **Purpose:** Allows applications to send/receive data over the Internet.
 - **Two types of sockets:**
 - **TCP sockets:** Reliable, connection-oriented (used for HTTP, FTP, email).
 - **UDP sockets:** Fast, connectionless, no guarantees (used for gaming, streaming).
 - **Example:** Writing a simple chat app using sockets.
-

Summary

- Application layer enables Internet services like web, email, streaming.
 - Applications follow **client-server or P2P models**.
 - **Protocols (HTTP, SMTP, DNS)** define communication rules.
 - Different apps need different **transport-layer services (TCP/UDP)**.
 - **HTTP evolved** from non-persistent → persistent → HTTP/2 → HTTP/3.
 - **DNS** maps names to IPs, crucial for usability.
 - **CDNs and DASH** enable scalable video streaming.
 - **Socket programming** lets developers build real network applications.
-